

Name:P.Tejaswini
Rollno:2520030011

Bank database triggers and stored procedures

Program:

```
1  ● CREATE DATABASE BankDB;
2  ● USE BankDB;
3
4  ● ○ CREATE TABLE Customer (
5      Customer_ID INT PRIMARY KEY,
6      Customer_Name VARCHAR(100) NOT NULL,
7      Phone VARCHAR(15),
8      Email VARCHAR(100),
9      City VARCHAR(50)
10 ) ;
11
12 ● ○ CREATE TABLE Account (
13     Account_No INT PRIMARY KEY,
14     Customer_ID INT,
15     Account_Type VARCHAR(20),
16     Balance DECIMAL(12,2) DEFAULT 0,
17     Branch VARCHAR(50),
18     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
19 ) ;
20
21 ● ○ CREATE TABLE Bank_Transaction (
22     Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
23     Account_No INT,
24     Transaction_Type VARCHAR(20),
25     Amount DECIMAL(12,2),
26     Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,
27     FOREIGN KEY (Account_No) REFERENCES Account(Account_No)
28 ) ;
```

Output:

Output					
Action Output					
#	Time	Action	Message	Duration / Fetch	
✓ 1	10:20:28	CREATE DATABASE BankDB	1 row(s) affected	0.031 sec	
✓ 2	10:20:31	USE BankDB	0 row(s) affected	0.000 sec	
✓ 3	10:20:32	CREATE TABLE Customer (Customer_ID INT PR...	0 row(s) affected	0.062 sec	
✓ 4	10:20:35	CREATE TABLE Account (Account_No INT PRI...	0 row(s) affected	0.062 sec	
✓ 5	10:20:51	CREATE TABLE Bank_Transaction (Transaction...	0 row(s) affected	0.094 sec	

Program:

```
30 ● CREATE TABLE Loan (  
31     Loan_ID INT PRIMARY KEY,  
32     Customer_ID INT,  
33     Loan_Type VARCHAR(30),  
34     Loan_Amount DECIMAL(12,2),  
35     Interest_Rate DECIMAL(5,2),  
36     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)  
37 );  
38 ● INSERT INTO Customer (Customer_ID, Customer_Name, Phone, Email, City) VALUES  
39     (101, 'Tejaswini', '9876543210', 'tejaswini@gmail.com', 'Hyderabad'),  
40     (102, 'Priya Sharma', '9876543211', 'priya@gmail.com', 'Vijayawada'),  
41     (103, 'Arjun Reddy', '9876543212', 'arjun@gmail.com', 'Bangalore'),  
42     (104, 'Sneha Rao', '9876543213', 'sneha@gmail.com', 'Chennai'),  
43     (105, 'Kiran Kumar', '9876543214', 'kiran@gmail.com', 'Hyderabad');  
44 ● INSERT INTO Account (Account_No, Customer_ID, Account_Type, Balance, Branch) VALUES  
45     (10001, 101, 'Savings', 50000, 'Hyderabad'),  
46     (10002, 102, 'Savings', 75000, 'Vijayawada'),  
47     (10003, 103, 'Current', 120000, 'Bangalore'),  
48     (10004, 104, 'Savings', 45000, 'Chennai'),  
49     (10005, 105, 'Current', 90000, 'Hyderabad');  
50 ● INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES  
51     (10001, 'DEPOSIT', 10000),  
52     (10002, 'DEPOSIT', 15000),  
53     (10003, 'WITHDRAW', 20000),  
54     (10004, 'DEPOSIT', 5000),  
55     (10005, 'WITHDRAW', 10000);
```

Output:

✓ 6	10:22:05	CREATE TABLE Loan (Loan_ID INT PRIMARY...	0 row(s) affected	0.078 sec	
✓ 7	10:22:08	INSERT INTO Customer (Customer_ID, Customer_...	5 row(s) affected Records: 5 Duplicates: 0 Wamin...	0.047 sec	
✓ 8	10:22:27	INSERT INTO Account (Account_No, Customer_J...	5 row(s) affected Records: 5 Duplicates: 0 Wamin...	0.031 sec	
✓ 9	10:22:45	INSERT INTO Bank_Transaction (Account_No, Tr...	5 row(s) affected Records: 5 Duplicates: 0 Wamin...	0.016 sec	

Program:

```
56 ● INSERT INTO Loan (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate) VALUES
57     (501,101,'Home Loan',5000000,7.5),
58     (502,102,'Education Loan',1000000,6.5),
59     (503,103,'Car Loan',800000,8.2),
60     (504,104,'Personal Loan',500000,10.5);
61
62 ● SELECT * FROM Customer;
```

Output:

The screenshot displays a database management interface. At the top, a table titled 'Customer' is shown with the following data:

Customer_ID	Customer_Name	Phone	Email	City
101	Tejaswini	9876543210	tejaswini@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
NULL	NULL	NULL	NULL	NULL

Below the table, the 'Output' section shows a log of database actions:

#	Time	Action	Message	Duration / Fetch
6	10:22:05	CREATE TABLE Loan (Loan_ID INT PRIMAR...	0 row(s) affected	0.078 sec
7	10:22:08	INSERT INTO Customer (Customer_ID, Customer...	5 row(s) affected Records: 5 Duplicates: 0 Wami...	0.047 sec
8	10:22:27	INSERT INTO Account (Account_No, Customer_...	5 row(s) affected Records: 5 Duplicates: 0 Wami...	0.031 sec
9	10:22:45	INSERT INTO Bank_Transaction (Account_No, ...	5 row(s) affected Records: 5 Duplicates: 0 Wami...	0.016 sec
10	10:23:41	INSERT INTO Loan (Loan_ID, Customer_ID, Loa...	4 row(s) affected Records: 4 Duplicates: 0 Wami...	0.016 sec
11	10:23:48	SELECT * FROM Customer LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

Program:

```
63 ● SELECT * FROM Account;
```

Output:

Result Grid						Filter Rows:	Edit:	Export/Import
	Account_No	Customer_ID	Account_Type	Balance	Branch			
▶	10001	101	Savings	50000.00	Hyderabad			
	10002	102	Savings	75000.00	Vijayawada			
	10003	103	Current	120000.00	Bangalore			
	10004	104	Savings	45000.00	Chennai			
	10005	105	Current	90000.00	Hyderabad			
*	NULL	NULL	NULL	NULL	NULL			

12 10:24:42 SELECT * FROM Account LIMIT 0, 1000 5 row(s) returned 0.000 sec / 0.000 sec

Program:

```
64 • SELECT * FROM Bank_Transaction;
```

Output:

Result Grid						Filter Rows:	Edit:	Export/Import
	Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date			
▶	1	10001	DEPOSIT	10000.00	2026-08-15 10:22:4			
	2	10002	DEPOSIT	15000.00	2026-08-15 10:22:4			
	3	10003	WITHDRAW	20000.00	2026-08-15 10:22:4			
	4	10004	DEPOSIT	5000.00	2026-08-15 10:22:4			
	5	10005	WITHDRAW	10000.00	2026-08-15 10:22:4			
*	NULL	NULL	NULL	NULL	NULL			

13 10:25:22 SELECT * FROM Bank_Transaction LIMIT 0, 1000 5 row(s) returned 0.000 sec / 0.000 sec

Program:

```
65 • SELECT * FROM Loan;
```

Output:

	Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
▶	501	101	Home Loan	5000000.00	7.50
	502	102	Education Loan	1000000.00	6.50
	503	103	Car Loan	800000.00	8.20
	504	104	Personal Loan	500000.00	10.50
*	NULL	NULL	NULL	NULL	NULL

14 10:26:39 SELECT * FROM Loan LIMIT 0, 1000 4 row(s) returned 0.000 sec / 0.000 sec

Program:

```
69 CREATE PROCEDURE GetAllCustomers()  
70 BEGIN  
71     SELECT * FROM Customer;  
72 END //  
73  
74 DELIMITER ;  
75
```

Output:

✓ 16 10:27:48 CREATE PROCEDURE GetAllCustomers() BEGIN... 0 row(s) affected 0.047 sec

Program:

```
76 CALL GetAllCustomers();
```

Output:

	Customer_ID	Customer_Name	Phone	Email	City
▶	101	Tejaswini	9876543210	tejaswini@gmail.com	Hyderabad
	102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
	103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
	104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
	105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad

✓ 18 10:28:11 CALL GetAllCustomers() 5 row(s) returned 0.031 sec / 0.000 sec

Program:

```
80 CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT)  
81 BEGIN  
82     SELECT *  
83     FROM Account  
84     WHERE Account_No=p_Account_No;  
85 END //  
86  
87 DELIMITER ;  
88
```

Output:

✓ 19 10:29:45 CREATE PROCEDURE GetAccountDetails(IN p_... 0 row(s) affected 0.016 sec

Program:

```
89 CALL GetAccountDetails(10001);  
90  
91 DELIMITER //
```

Output:

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	50000.00	Hyderabad

✓	20	10:30:25	CALL GetAccountDetails(10001)	1 row(s) returned	0.031 sec / 0.000 sec
---	----	----------	-------------------------------	-------------------	-----------------------

Program:

```
93 ● CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT)
94 BEGIN
95     SELECT
96         C.Customer_ID,
97         C.Customer_Name,
98         A.Account_No,
99         A.Account_Type,
100         A.Balance,
101         A.Branch
102     FROM Customer C
103     JOIN Account A ON C.Customer_ID=A.Customer_ID
104     WHERE C.Customer_ID=p_Customer_ID;
105 END //
106
107 DELIMITER ;
108
```

Output:

✓	21	10:31:18	CREATE PROCEDURE GetCustomerAccounts(l...	0 row(s) affected	0.047 sec
---	----	----------	---	-------------------	-----------

Program:

```
109 ● CALL GetCustomerAccounts(101);
```

Output:

	Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Bran
▶	101	Tejaswini	10001	Savings	50000.00	Hyde

✓	22	10:32:17	CALL GetCustomerAccounts(101)	1 row(s) returned	0.031 sec / 0.000 sec
---	----	----------	-------------------------------	-------------------	-----------------------

Program:

```
113 ● ○ CREATE PROCEDURE DepositMoney(  
114     IN p_Account_No INT,  
115     IN p_Amount DECIMAL(12,2)  
116 )  
117 ○ BEGIN  
118     UPDATE Account  
119     SET Balance=Balance+p_Amount  
120     WHERE Account_No=p_Account_No;  
121 END //  
122  
123 DELIMITER ;  
124
```

Output:

✓	23	10:33:09	CREATE PROCEDURE DepositMoney(IN p_A...	0 row(s) affected	0.047 sec
---	----	----------	--	-------------------	-----------

Program:

```
125 ● CALL DepositMoney(10001,5000);  
126  
127 DELIMITER //
```

Output:

✓	24	10:33:47	CALL DepositMoney(10001,5000)	1 row(s) affected	0.031 sec
---	----	----------	-------------------------------	-------------------	-----------

Program:

```
129 ● ○ CREATE PROCEDURE WithdrawMoney(  
130     IN p_Account_No INT,  
131     IN p_Amount DECIMAL(12,2)  
132 )  
133 ○ BEGIN  
134     UPDATE Account  
135     SET Balance=Balance-p_Amount  
136     WHERE Account_No=p_Account_No;  
137 END //  
138  
139 DELIMITER ;  
140
```

Output:

✓	25	10:34:19	CREATE PROCEDURE WithdrawMoney(IN p...	0 row(s) affected	0.031 sec
---	----	----------	---	-------------------	-----------

Program:

```
141 ● CALL WithdrawMoney(10001,3000);
142
143 DELIMITER //
144
```

Output:

✓	26	10:34:56	CALL WithdrawMoney(10001,3000)	1 row(s) affected	0.000 sec
---	----	----------	--------------------------------	-------------------	-----------

Program:

```
145 ● CREATE TRIGGER CheckBalance
146 BEFORE UPDATE ON Account
147 FOR EACH ROW
148 BEGIN
149 IF NEW.Balance<0 THEN
150     SIGNAL SQLSTATE '45000'
151     SET MESSAGE_TEXT='Transaction failed: Insufficient balance';
152 END IF;
153 END //
154
155 DELIMITER ;
156
```

Output:

✓	27	10:35:38	CREATE TRIGGER CheckBalance BEFORE UP...	0 row(s) affected	0.031 sec
---	----	----------	--	-------------------	-----------

Program:

```
157 ● UPDATE Account
158 SET Balance=Balance-60000
159 WHERE Account_No=10001;
160
161 DELIMITER //
```

Output:

✗	28	10:36:15	UPDATE Account SET Balance=Balance-60000 ...	Error Code: 1644. Transaction failed: Insufficient b...	0.000 sec
---	----	----------	--	---	-----------

Program:

```
163 ● CREATE TRIGGER CheckTransactionAmount
164 BEFORE INSERT ON Bank_Transaction
165 FOR EACH ROW
166 BEGIN
167     IF NEW.Amount <= 0 THEN
168         SIGNAL SQLSTATE '45000'
169         SET MESSAGE_TEXT='Transaction amount must be greater than zero';
170     END IF;
171 END //
172
173 DELIMITER ;
174
```

Output:

✓	29	10:36:46	CREATE TRIGGER CheckTransactionAmount B...	0 row(s) affected	0.031 sec
---	----	----------	--	-------------------	-----------

Program:

```
175 ● INSERT INTO Bank_Transaction
176 (Account_No, Transaction_Type, Amount)
177 VALUES
178 (10001, 'DEPOSIT', -5000);
179
```

Output:

✗	30	10:37:26	INSERT INTO Bank_Transaction (Account_No, T...	Error Code: 1644. Transaction amount must be gr...	0.000 sec
---	----	----------	--	--	-----------

Program:

```
180 ● CREATE TABLE Transaction_Audit (
181     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
182     Transaction_ID INT,
183     Account_No INT,
184     Transaction_Type VARCHAR(20),
185     Amount DECIMAL(12,2),
186     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
187 );
188
189 DELIMITER //
190
```

Output:

✓	31	10:38:06	CREATE TABLE Transaction_Audit (Audit_ID ...	0 row(s) affected	0.047 sec
---	----	----------	---	-------------------	-----------

Program:

```
191 ● CREATE TRIGGER TransactionAudit
192 AFTER INSERT ON Bank_Transaction
193 FOR EACH ROW
194 BEGIN
195     INSERT INTO Transaction_Audit
196     (Transaction_ID,Account_No,Transaction_Type,Amount)
197     VALUES
198     (NEW.Transaction_ID,NEW.Account_No,NEW.Transaction_Type,NEW.Amount);
199 END //
200
201 DELIMITER ;
202
```

Output:

✓	32	10:38:41	CREATE TRIGGER TransactionAudit AFTER IN...	0 row(s) affected	0.031 sec
---	----	----------	---	-------------------	-----------

Program:

```
203 ● INSERT INTO Bank_Transaction
204 (Account_No,Transaction_Type,Amount)
205 VALUES
206 (10001,'DEPOSIT',2500);
207
```

Output:

✓	33	10:39:22	INSERT INTO Bank_Transaction (Account_No,T...	1 row(s) affected	0.031 sec
---	----	----------	---	-------------------	-----------

Program:

```
208 ● SELECT * FROM Transaction_Audit;
```

Output:

	Audit_ID	Transaction_ID	Account_No	Transaction_Type	Amount	Audit_D
▶	1	6	10001	DEPOSIT	2500.00	2026-08
*	NULL	NULL	NULL	NULL	NULL	NULL

✓	34	10:39:50	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
---	----	----------	---	-------------------	-----------------------

Program:

```
212 ● CREATE TRIGGER UpdateBalanceAfterTransaction
213 AFTER INSERT ON Bank_Transaction
214 FOR EACH ROW
215 BEGIN
216     IF NEW.Transaction_Type='DEPOSIT' THEN
217         UPDATE Account
218         SET Balance=Balance+NEW.Amount
219         WHERE Account_No=NEW.Account_No;
220     ELSEIF NEW.Transaction_Type='WITHDRAW' THEN
221         UPDATE Account
222         SET Balance=Balance-NEW.Amount
223         WHERE Account_No=NEW.Account_No;
224     END IF;
225 END //
226
227 DELIMITER ;
228
```

Output:

✓	35	10:40:40	CREATE TRIGGER UpdateBalanceAfterTransact...	0 row(s) affected	0.016 sec
---	----	----------	--	-------------------	-----------

Program:

```
229 ● INSERT INTO Bank_Transaction
230 (Account_No,Transaction_Type,Amount)
231 VALUES
232 (10001,'DEPOSIT',5000);
```

Output:

✓	36	10:41:17	INSERT INTO Bank_Transaction (Account_No,T...	1 row(s) affected	0.015 sec
---	----	----------	---	-------------------	-----------

Program:

```
234 ● SELECT *
235 FROM Account
236 WHERE Account_No=10001;
237
238 DELIMITER //
239
```

Output:

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	57000.00	Hyderabad
•	NULL	NULL	NULL	NULL	NULL

✓ 37 10:41:44 SELECT * FROM Account WHERE Account_No... 1 row(s) returned 0.000 sec / 0.000 sec

Program:

```
240 ● CREATE TRIGGER PreventInsufficientWithdrawal
241 BEFORE INSERT ON Bank_Transaction
242 FOR EACH ROW
243 BEGIN
244     DECLARE CurrentBalance DECIMAL(12,2);
245
246     SELECT Balance
247     INTO CurrentBalance
248     FROM Account
249     WHERE Account_No=NEW.Account_No;
250
251     IF NEW.Transaction_Type='WITHDRAW'
252     AND NEW.Amount>CurrentBalance THEN
253         SIGNAL SQLSTATE '45000'
254         SET MESSAGE_TEXT='Withdrawal failed: Insufficient balance';
255     END IF;
256 END //
257
258 DELIMITER ;
259
```

Output:

✓ 38 10:42:40 CREATE TRIGGER PreventInsufficientWithdraw... 0 row(s) affected 0.032 sec

Program:

```
260 ● INSERT INTO Bank_Transaction
261 (Account_No,Transaction_Type,Amount)
262 VALUES
263 (10001,'WITHDRAW',1000000);
264
265 DELIMITER //
266
```

Output:

✗ 39 10:43:18 INSERT INTO Bank_Transaction (Account_No,T... Error Code: 1644. Withdrawal failed: Insufficient b... 0.016 sec

Program:

```
267 CREATE PROCEDURE TransferMoney(  
268     IN SenderAccount INT,  
269     IN ReceiverAccount INT,  
270     IN TransferAmount DECIMAL(12,2)  
271 )  
272 BEGIN  
273     DECLARE SenderBalance DECIMAL(12,2);  
274  
275     SELECT Balance  
276     INTO SenderBalance  
277     FROM Account  
278     WHERE Account_No=SenderAccount;  
279  
280     IF SenderBalance<TransferAmount THEN  
281         SIGNAL SQLSTATE '45000'  
282         SET MESSAGE_TEXT='Transfer failed: Insufficient balance';  
283     ELSE  
284         UPDATE Account  
285         SET Balance=Balance-TransferAmount  
286         WHERE Account_No=SenderAccount;  
287  
288         UPDATE Account  
289         SET Balance=Balance+TransferAmount  
290         WHERE Account_No=ReceiverAccount;  
291     END IF;  
292 END //  
293  
294 DELIMITER ;  
295
```

Output:

✓	40	10:44:00	CREATE PROCEDURE TransferMoney(IN Se...	0 row(s) affected	0.031 sec
---	----	----------	--	-------------------	-----------

Program:

```
296 CALL TransferMoney(10001,10002,5000);
```

Output:

✓	41	10:44:47	CALL TransferMoney(10001,10002,5000)	1 row(s) affected	0.031 sec
---	----	----------	--------------------------------------	-------------------	-----------

Program:

```
298 ● SELECT *
299     FROM Account
300     WHERE Account_No IN (10001,10002);
301
302     DELIMITER //
303
```

Output:

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	52000.00	Hyderabad
	10002	102	Savings	80000.00	Vijayawada
✱	NULL	NULL	NULL	NULL	NULL

✓ 42 10:45:28 SELECT * FROM Account WHERE Account_No ... 2 row(s) returned 0.000 sec / 0.000 sec

Program:

```
304 ● CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT)
305     BEGIN
306         SELECT
307             C.Customer_Name,
308             L.Loan_ID,
309             L.Loan_Type,
310             L.Loan_Amount,
311             L.Interest_Rate
312         FROM Customer C
313         JOIN Loan L ON C.Customer_ID=L.Customer_ID
314         WHERE C.Customer_ID=p_Customer_ID;
315     END //
316
317     DELIMITER ;
318
```

Output:

✓ 43 10:46:38 CREATE PROCEDURE GetCustomerLoans(IN p_... 0 row(s) affected 0.031 sec

Program:

```
319 ● CALL GetCustomerLoans(101);
```

Output:

	Customer_Name	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
▶	Tejaswini	501	Home Loan	5000000.00	7.50

✓ 44 10:46:54 CALL GetCustomerLoans(101) 1 row(s) returned 0.000 sec / 0.000 sec

Program:

```
323 ● ○ CREATE PROCEDURE HighBalanceAccounts(  
324     IN MinimumBalance DECIMAL(12,2)  
325 )  
326 ○ BEGIN  
327     SELECT *  
328     FROM Account  
329     WHERE Balance >= MinimumBalance  
330     ORDER BY Balance DESC;  
331 END //  
332  
333 DELIMITER ;  
334
```

Output:

✓ 45 10:48:01 CREATE PROCEDURE HighBalanceAccounts(... 0 row(s) affected 0.000 sec

Program:

```
335 ● CALL HighBalanceAccounts(50000);  
336  
337 DELIMITER //  
338
```

Output:

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10003	103	Current	120000.00	Bangalore
	10005	105	Current	90000.00	Hyderabad
	10002	102	Savings	80000.00	Vijayawada
	10001	101	Savings	52000.00	Hyderabad

✓ 46 10:48:18 CALL HighBalanceAccounts(50000) 4 row(s) returned 0.000 sec / 0.000 sec

Program:

```
339 ● ○ CREATE PROCEDURE GetBalance(  
340     IN p_Account_No INT,  
341     OUT p_Balance DECIMAL(12,2)  
342 )  
343 ○ BEGIN  
344     SELECT Balance  
345     INTO p_Balance  
346     FROM Account  
347     WHERE Account_No=p_Account_No;  
348     END //  
349  
350     DELIMITER ;  
351
```

Output:

✓	47	10:49:43	CREATE PROCEDURE GetBalance(IN p_Acc...	0 row(s) affected	0.015 sec
---	----	----------	--	-------------------	-----------

Program:

```
352 ● CALL GetBalance(10001,@CurrentBalance);  
353 ● SELECT @CurrentBalance;
```

Output:

✓	48	10:50:01	CALL GetBalance(10001,@CurrentBalance)	1 row(s) affected	0.032 sec
---	----	----------	--	-------------------	-----------

Program:

```
355 ● SHOW TRIGGERS;
```


Output:

	Trigger	Event	Table	Statement
▶	CheckBalance	UPDATE	account	BEGIN IF NEW.Bal
	CheckTransactionAmount	INSERT	bank_transaction	BEGIN IF NEW.Am
	PreventInsufficientWithdrawal	INSERT	bank_transaction	BEGIN DECLARE C
	TransactionAudit	INSERT	bank_transaction	BEGIN INSERT INT
	UpdateBalanceAfterTransaction	INSERT	bank_transaction	BEGIN IF NEW.Tra

Result 14 × Read Only

✓ 49 10:50:29 SHOW TRIGGERS 5 row(s) returned 0.031 sec / 0.000 sec

Program:

```
357 ● SHOW CREATE TRIGGER CheckBalance;
358
```

Output:

	Trigger	sql_mode	SQL Original Statement
▶	CheckBalance	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	CREATE DEFINER=

Result Grid

✓ 50 10:51:22 SHOW CREATE TRIGGER CheckBalance 1 row(s) returned 0.016 sec / 0.000 sec

Program:

```
359 ● DROP TRIGGER CheckBalance;
360
361 ● SHOW PROCEDURE STATUS
362 WHERE Db= 'BankDB';
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Db	Name	Type	Definer	Modified
▶	bankdb	DepositMoney	PROCEDURE	root@localhost	2026-08-15 10:3
	bankdb	GetAccountDetails	PROCEDURE	root@localhost	2026-08-15 10:2
	bankdb	GetAllCustomers	PROCEDURE	root@localhost	2026-08-15 10:2
	bankdb	GetBalance	PROCEDURE	root@localhost	2026-08-15 10:4
	bankdb	GetCustomerAccounts	PROCEDURE	root@localhost	2026-08-15 10:3
	bankdb	GetCustomerLoans	PROCEDURE	root@localhost	2026-08-15 10:4
	bankdb	HighBalanceAccounts	PROCEDURE	root@localhost	2026-08-15 10:4
	bankdb	TransferMoney	PROCEDURE	root@localhost	2026-08-15 10:4
	bankdb	WithdrawMoney	PROCEDURE	root@localhost	2026-08-15 10:4

Result 16 x | Read Only | Context Help | Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 47	10:49:43	CREATE PROCEDURE GetBalance(IN p_Acc...	0 row(s) affected	0.015 sec
✓ 48	10:50:01	CALL GetBalance(10001,@CurrentBalance)	1 row(s) affected	0.032 sec
✓ 49	10:50:29	SHOW TRIGGERS	5 row(s) returned	0.031 sec / 0.000 sec
✓ 50	10:51:22	SHOW CREATE TRIGGER CheckBalance	1 row(s) returned	0.016 sec / 0.000 sec
✓ 51	10:52:03	DROP TRIGGER CheckBalance	0 row(s) affected	0.047 sec
✓ 52	10:52:10	SHOW PROCEDURE STATUS WHERE Db='Ba...	9 row(s) returned	0.032 sec / 0.000 sec

Program:

```
364 ● DROP PROCEDURE GetAllCustomers;
```

Output:

✓ 53	10:52:48	DROP PROCEDURE GetAllCustomers	0 row(s) affected	0.032 sec
------	----------	--------------------------------	-------------------	-----------

Practice Questions:

a) Procedures Concept:

QUESTION 1:

Create a procedure to display all accounts belonging to a particular branch.

Example:

CALL GetBranchAccounts('Hyderabad');

Program:

```
368  /* Practice questions*/
369  CREATE PROCEDURE GetBranchAccounts(IN p_Branch VARCHAR(50))
370  BEGIN
371      SELECT *
372      FROM Account
373      WHERE Branch = p_Branch;
374  END //
375
376  DELIMITER ;
377
```

Output:

54 10:58:00 CREATE PROCEDURE GetBranchAccounts(IN p_Branch VARCHAR(50)) BEGIN SELECT * FROM Account... 0 row(s) affected 0.047 sec

Program:

```
378  CALL GetBranchAccounts('Hyderabad');
```

Output:

Result Grid					
Filter Rows:					
Export:  Wrap Cell Content: 					
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	52000.00	Hyderabad
	10005	105	Current	90000.00	Hyderabad

55 11:02:43 CALL GetBranchAccounts('Hyderabad') 2 row(s) returned 0.016 sec / 0.000 sec

QUESTION 2:

Create a procedure that accepts Customer_ID and displays:

Customer Name

Phone

Email

Account Number

Account Type

Balance

Program:

```
382 ● CREATE PROCEDURE GetCustomerAccountDetails(IN p_Customer_ID INT)
383 BEGIN
384     SELECT
385         C.Customer_Name,
386         C.Phone,
387         C.Email,
388         A.Account_No,
389         A.Account_Type,
390         A.Balance
391     FROM Customer C
392     JOIN Account A
393     ON C.Customer_ID = A.Customer_ID
394     WHERE C.Customer_ID = p_Customer_ID;
395 END //
396
397 DELIMITER ;
398
399 ● CALL GetCustomerAccountDetails(101);
```

Output:

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Customer_Name	Phone	Email	Account_No	Account_Ty
	Tejaswini	9876543210	tejaswini@gmail.com	10001	Savings

56

11:04:35

CREATE PROCEDURE GetCustomerAccountDet...

0 row(s) affected

0.047 sec

57

11:04:55

CALL GetCustomerAccountDetails(101)

1 row(s) returned

0.000 sec / 0.000 sec

QUESTION 3:

Create a procedure to calculate the total balance of all accounts belonging to a particular customer.

Program:

```
402 ● CREATE PROCEDURE GetTotalBalance(IN p_Customer_ID INT)
403 BEGIN
404     SELECT
405         C.Customer_ID,
406         C.Customer_Name,
407         SUM(A.Balance) AS Total_Balance
408     FROM Customer C
409     JOIN Account A
410     ON C.Customer_ID = A.Customer_ID
411     WHERE C.Customer_ID = p_Customer_ID
412     GROUP BY C.Customer_ID, C.Customer_Name;
413 END //
414
415 DELIMITER ;
416
417 ● CALL GetTotalBalance(101);
```

Output:

Result Grid		Filter Rows:	Export:
	Customer_ID	Customer_Name	Total_Balance
▶	101	Tejaswini	52000.00

✓	58	11:06:47	CREATE PROCEDURE GetTotalBalance(IN p_C...	0 row(s) affected	0.031 sec
✓	59	11:06:49	CALL GetTotalBalance(101)	1 row(s) returned	0.015 sec / 0.000 sec

b) Triggers Concept

QUESTION 5:

Create a trigger that prevents an account balance from becoming negative.

Program:

```
419 DELIMITER //
420
421 ● CREATE TRIGGER PreventNegativeBalance
422 BEFORE UPDATE ON Account
423 FOR EACH ROW
424 BEGIN
425     IF NEW.Balance < 0 THEN
426         SIGNAL SQLSTATE '45000'
427         SET MESSAGE_TEXT = 'Balance cannot be negative';
428     END IF;
429 END //
430
431 DELIMITER ;
```

Output:

✓	60	11:09:20	CREATE TRIGGER PreventNegativeBalance BE...	0 row(s) affected	0.000 sec
---	----	----------	---	-------------------	-----------

QUESTION 6:

Create a trigger that prevents a transaction amount from being zero or negative.

Program:

```
433 DELIMITER //
434
435 ● CREATE TRIGGER PreventInvalidTransaction
436 BEFORE INSERT ON Bank_Transaction
437 FOR EACH ROW
438 BEGIN
439     IF NEW.Amount <= 0 THEN
440         SIGNAL SQLSTATE '45000'
441         SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';
442     END IF;
443 END //
444
445 DELIMITER ;
```

Output:

✓	61	11:11:13	CREATE TRIGGER PreventInvalidTransaction B...	0 row(s) affected	0.032 sec
---	----	----------	---	-------------------	-----------

QUESTION 7:

Create a trigger that automatically records every transaction in Transaction_Audit.

Program:

```
448 DELIMITER //
449
450 ● CREATE TRIGGER AuditTransaction
451 AFTER INSERT ON Bank_Transaction
452 FOR EACH ROW
453 BEGIN
454     INSERT INTO Transaction_Audit
455     (Transaction_ID, Account_No, Transaction_Type, Amount)
456     VALUES
457     (NEW.Transaction_ID, NEW.Account_No, NEW.Transaction_Type, NEW.Amount);
458 END //
459
460 DELIMITER ;
```

Output:

✓	62	11:12:29	CREATE TRIGGER AuditTransaction AFTER IN...	0 row(s) affected	0.046 sec
---	----	----------	---	-------------------	-----------